

AIPL / ABCL^c+ User’s Guide

— syntax, usage, and sample programs —

ABCL^c+ / AIOS implementation (yaskodama/abclcp, branch `make-src-base`)

2026-08-23 edition

Target: branch `make-src-base` (the current specification, including the seven disciplines)

Abstract

This is the **current** user’s guide to AIPL (ABCL^c+). Part I gives the shape of the language and the shortest path to running something; Part II specifies the syntax and the type system; Part III covers the seven disciplines that sit on top of it; Part V walks through the sample programs with their sources and their actual output; Part VI describes the tools for building, running and checking; Part VII collects the grammar summary, the present limits, and the migration notes from the previous edition.

AIPL is a concurrent language whose unit is the **actor**. A value is not returned with **return** but with **reply**. Three static mechanisms sit on top of that — **types** (the return type is inferred from **reply**, and can also be annotated), **effects** (a method declares what it does to the world, as in `!{ai, net}`), and **deadlines** (`now ... timeout ... else ...` bounds every wait). For example, “a real-time actor synchronously calls a model outside the box” can be forbidden by the effect check alone.

Above those sit the **seven disciplines** (Part III) — the failure type `result< $\tau$ >`, first-class reply destinations, resource ordering, obligation levels, and session types. All of them are built so that they **work without you writing annotations**. As a result, **as long as the target of a wait is known as a class, or the level of the target node is declared, deadlock freedom is guaranteed by the type system**. That is not a claim on paper only: it is machine-checked in Coq/Rocq.

What this guide describes is the **core** — the part that all three implementations (OCaml, Python, JavaScript) provide. The Python implementation additionally has **extensions** (record types, refinement types, channels, ownership), so the language has two layers (§43).

If you only want the delta from the previous edition (2026-07-30), read §43. **become** has been removed from the language.

Contents

I	Getting started	1
1	What AIPL is	1
2	Three static mechanisms	2
2.1	Types — what you give back	2
2.2	Effects — what you do to the world	2
2.3	Deadlines — how long you will wait	2
2.4	And seven disciplines on top	2
3	Running something in five minutes	3
3.1	Building the implementation	3
3.2	Running a program	3
3.3	Running only the type checker	3

II	Language guide	3
4	The shape of a program	3
5	Lexical structure	4
6	Types	5
7	Expressions	5
7.1	Precedence	5
7.2	Do not confuse + with ++	6
7.3	Ambiguous overloads	6
8	Statements	6
9	Actors and communication	6
9.1	The three sends	6
9.2	reply	7
9.3	select	7
10	Waiting with a deadline	7
10.1	Why it is needed	7
10.2	The rules	7
10.3	Relation to type soundness	8
11	Effects	8
11.1	The eight effects	8
11.2	How to write them	8
11.3	What counts as an effect	9
11.4	What this lets you forbid	9
III	The seven disciplines	9
12	Putting failure in the type — $\text{result}\langle\tau\rangle$	10
13	Making the reply destination a first-class value	10
14	Resource order	11
14.1	A global order — stopping reverse acquisition	11
15	Obligation levels — stopping cycles of waiting	12
15.1	Waits that cross nodes	12
15.2	How much is guaranteed	12
16	Session types — the order of an exchange	12
16.1	When the session crosses actors	13
17	The select discipline	13
18	Mesh deployment	13
19	Escape hatches (environment variables)	14
IV	Reference	14

20 What the type checker looks at	14
21 Exposing to the outside	15
22 Builtin functions and their effects	15
 V Sample programs	 16
23 g1: classes, fields, send, string concatenation	16
24 g2: now / future / await and annotations	16
25 g3: several actors, and a now across actors	17
26 g4: select, and which message a reply answers	18
27 g5: exposing to the outside makes annotations mandatory	18
28 g6: effect annotations and propagation	19
29 g7: now / await with deadlines	20
30 g8: equality, unary minus, boolean literals	21
31 g9: passing an actor as an argument	22
32 g10: two classes with a field of the same name	22
33 Negative samples	23
 VI Building and running	 23
34 Building the implementation	23
35 Running a program	23
36 Running only the type checker	24
37 The type/run-time differential test	24
38 Environment variables	24
 VII Appendix	 24
39 Grammar summary	24
40 Environment variables of the implementation	25
41 Common pitfalls	26
42 Present limits	27

43 Changes from the previous edition	27
43.1 Removed from the language	27
43.2 Added	27
43.3 Made stricter	28
43.4 The file extension	28
43.5 Core and extensions — the language has two layers	28
44 Related documents	28

Part I

Getting started

1 What AIPL is

AIPL is an actor language in the lineage of ABCL/1. A program is a sequence of **class declarations** and **global statements**; actors created from the classes run by sending messages to each other.

```
class Counter {
  var n = 0;                // a field (the actor's state)
  method bump() : int !{mut} { // return type int, effect mut
    n = n + 1;
    reply(n);               // hand a value back to the caller
  }
}

var c = new Counter();      // create one actor
print(now c.bump() timeout 100 else 0);
```

Four differences from other languages are worth learning first.

1. **One actor is one thread**, and it processes the messages it receives one at a time. Fields are never touched concurrently.
2. **Values come back through reply**. There is no **return**. **reply** resolves the caller's future; the method keeps running afterwards.
3. **There are three kinds of send**. **send** (does not wait; a statement), **now** (waits for the reply; an expression), and **future** (does not wait; hands you a ticket; an expression).
4. **Waiting freezes you**. While an actor waits in a **now** it cannot process any other message. That is why waits carry deadlines.

2 Three static mechanisms

2.1 Types — what you give back

A method's return type is inferred from the **reply** calls in its body. The optional annotation **method** **m(x) : T** may also be written. When you write it, the checker verifies that every execution path replies, and the declared type flows into expression inference as the *expected* type.

2.2 Effects — what you do to the world

A method declares its effects with $\{ \dots \}$. There are eight of them (§11). If you omit the annotation the effects are inferred; if you write it, “the effects of the body $\subseteq$ the effects you declared” is checked.

```
method plan(goal) : string !{ai, net} { reply(ai_call("plan: " ++ goal)); }
method peek()      : int      !{}      { reply(n); }
```

The point is that `ai` and `net` are separate: on-device inference that “uses AI but never leaves the box” is expressed with `{ai}` alone.

2.3 Deadlines — how long you will wait

```
var r = now planner.plan(g) timeout 2000 else "(timed out)";
```

If no reply arrives in time, the value of the `else` branch is used. If you write no `else`, the expression has type `result( $\tau$ )` and cannot be used before it is checked (§12).

2.4 And seven disciplines on top

Types, effects and deadlines are the ground floor. Above them sit the failure type, first-class reply destinations, resource ordering, obligation levels and session types (Part III). All of them are built so that they **work without annotations**. As a result, **as long as the target of a wait is known as a class, or the level of the target node is declared, deadlock freedom is guaranteed by the type system** (§15).

3 Running something in five minutes

3.1 Building the implementation

```
cd ~/aios/abclcp
make -C src thread_repl
```

Building with `dune` fails because `src/lexer.ml` is generated twice, so use `src/Makefile`.

3.2 Running a program

Pass a file of REPL commands with `-f`.

```
cat > run.repl <<'EOF'
load docs/samples/guide/g2_now_future.aipl
compile
EOF
./src/abclrepl_thread -q -f run.repl < /dev/null
```

```
twice = 43
awaited = 20
v=7
```

Do not pass a .aipl file to `-f` directly. It would be read line by line as REPL commands, the class definitions would never be loaded, and you would get `Actor xxx not found`. Always write `load` and `compile`.

3.3 Running only the type checker

The REPL depends on SDL, so a small separate driver exists for type checking alone (§36). Once built, `./tc file.aipl` prints the table of return types, effects and signatures.

```
$ ./tc docs/samples/guide/g6_effects.aipl
=== g6_effects.aipl ===
[OK] type check passed
[method return types (inferred from reply)]
```

```

Counter#bump -> int
Planner#plan -> string
ViaNow#run -> string
[method effects (inferred / declared)]
Counter#bump !{mut}
Planner#plan !{ai, net}
ViaNow#run !{ai, net}

```

Part II

Language guide

4 The shape of a program

A program is a **sequence of declarations**. A declaration is one of the following; the order is free, and names are resolved at run time.

Declaration	Meaning
<code>class C { ... }</code>	class definition
<code>var x = e;</code>	global variable
<code>var x = new C(args);</code>	create an actor
<code>send t.m(args);</code>	global asynchronous send
<code>send! t.m(args);</code>	the same, with the check relaxed
<code>f(args);</code>	call a builtin function

`if` and `while` cannot appear at global level. If you need control flow, put it inside a method and start it with `send`.

A class is a **sequence of fields** followed by a **sequence of methods**.

```

class Counter {
  var n = 0;           // fields come first; declare with var or float
  float ratio = 1.5;
  method init(start) { // init is called automatically by new
    n = start;
  }
  method bump() : int !{mut} {
    n = n + 1;
    reply(n);
  }
}

```

- Fields come **before** methods.
- A class needs **at least one method** (the grammar has no field-only class).
- A method named `init` is called automatically by `new C(args)`. If the number of arguments does not match, creation is skipped and `[Actor] x: no init; skipped` is printed.
- A field's type is fixed by its initializer. `var n = 0;` is `int`, so assigning a `float` or a `string` to it later is a type error.

5 Lexical structure

Kind	Content
Comments	// to end of line, /* ... */ (not nestable)
Keywords	class, method, var, float, new, send, send!, now, future, await, if, else, while, do, select, case, timeout, call, replyto, self, sender, remote, true, false
Integers	0, 123
Floats	1.5, 100. (a trailing dot alone makes it float)
Strings	"...", with \\ \" \n \t \r
Booleans	true, false
Identifiers	[A-Za-z_] [A-Za-z0-9_]*

`float` is a keyword, so it cannot be used freely as a variable or type name (it *can* be written as a return annotation, : `float`, which has its own grammar rule).

6 Types

Type	Description
<code>int</code> , <code>float</code> , <code>string</code> , <code>bool</code> , <code>unit</code>	base types
<code>T[]</code>	array (handled through the <code>array_*</code> builtins)
<code>future T</code>	the result of a <code>future</code> send; <code>await</code> takes the <code>T</code> out
<code>actor(C)</code>	an actor of class <code>C</code> ; the type of <code>new C(...)</code>
<code>any</code>	a boundary whose content is not statically guaranteed (<code>sender</code> , remote target)

A return annotation may be `int` / `float` / `string` / `bool` / `unit` / `any`, or a class name starting with a capital letter (an actor type). An unknown lowercase name is a syntax error: **unknown type name in return annotation.**

Method parameters are monomorphic. The type variables of a signature are shared by every call site, so calling the same method with `int` and with `string` conflicts. Polymorphic methods cannot be written.

There is **no implicit conversion** between `int` and `float`. If `var h = new Hello(5)`; feeds a `float` field you get a type error; write `new Hello(5.)`.

7 Expressions

Expression	Meaning
123, 1.5, "abc", true, false	literals
x	variable, field or parameter
- e	unary minus; lowers to <code>neg(e)</code> . <code>int/float</code>
a ++ b	string concatenation (both sides are stringified)
a + b, a - b, a * b	arithmetic. <code>int</code> with <code>int</code> gives <code>int</code> ; <code>float</code> with <code>float</code> gives <code>float</code>
a / b	division. Even <code>int</code> with <code>int</code> gives <code>float</code> (no integer division)
a == b, a != b	equality. <code>int/float/string/bool</code>
a > b, a < b, a >= b, a <= b	comparison; the result is <code>bool</code>
new C(args)	create an actor (effect {mem})
now t.m(args)	synchronous send: wait for the reply and become that value
now t.m(args) timeout n else e	synchronous send with a deadline
future t.m(args)	asynchronous send; returns <code>future T</code>
await e	wait for a future to resolve
await e timeout n else e'	<code>await</code> with a deadline
f(args)	call a builtin function
(e)	parentheses

The target `t` is a **variable holding an actor reference**, or `remote("host:port", "actorName")`. It is not a class name.

7.1 Precedence

From weakest to strongest:

equality == != < comparison > < >= <= < concatenation ++ < additive + - < multiplicative * /
< unary minus

So `1 + 2 == 3` reads as `(1 + 2) == 3`, and `"x=" ++ a + b` reads as `"x=" ++ (a + b)`. Every binary operator is **left associative** (equality included) — `1 == 2 == false` reads as `((1 == 2) == false)` and evaluates to `true`. Unary minus is a prefix, so it can be stacked (`- - 3` is `3`).

7.2 Do not confuse + with ++

`+` is **numeric only**. Mixing in a string gives no overload of `+` matches `(string, int)`.

```
print("count = " ++ n);      // correct
print("count = " + n);       // type error
```

`++` stringifies both sides first, so neither side has to be a string.

7.3 Ambiguous overloads

When both operands are unbound, `a + b` could be `int` or `float`: it has no principal type. That is an error. An annotation makes the expected type flow into the expression and settles it.

```
method add(a, b) { reply(a + b); } // error: ambiguous overload
method add(a, b) : int { reply(a + b); } // the annotation settles it
```

`AIOS_LAX_OVERLOAD=1` restores the old behaviour (warn and pick the default candidate). Equality `==` and `!=` have four candidates but all of them return `bool`, so they are never ambiguous (ambiguity is only reported when the *return types* disagree).

8 Statements

Statement	Meaning
<code>var x = e;</code>	variable declaration
<code>var x = new C(args);</code>	declaration that creates an actor
<code>x = e;</code>	assignment
<code>f(args);</code> <code>call f(args);</code>	call a builtin function
<code>send t.m(args);</code>	asynchronous send; does not wait for a reply
<code>send self.m(args);</code>	asynchronous send to yourself
<code>send sender.m(args);</code>	asynchronous send to the sender
<code>send! t.m(args);</code>	send with the check relaxed
<code>if (e) s if (e) s else s</code>	conditional (the parentheses are required)
<code>while e do s</code>	loop (no parentheses required)
<code>{ s ... }</code>	block
<code>select { ... }</code>	selective receive

`now self.m()` and `now sender.m()` **cannot be written** — the grammar forbids them. A `now` to yourself is a self-deadlock (you are busy running that very method, so nobody is left to produce the reply), and not being able to write it is exactly the safe side. If you want to factor out a computation such as inverse kinematics, put it in another actor and call `now kin.solve(...)`.

A variable declaration with a type annotation, `var x : T = e;`, does **not exist yet**. It is in the design notes but unimplemented.

9 Actors and communication

9.1 The three sends

	Waits?	Statement or expression	Inherits the callee's effects?
<code>send a.m(x);</code>	no	statement	no
<code>now a.m(x)</code>	until reply	expression (of the declared/inferred type)	yes
<code>future a.m(x)</code>	no	expression (<code>future T</code>)	no

“Only the side that waits carries the callee’s effects” is the rule of effect propagation (§11).

9.2 reply

A method does not “return” a value; `reply(v)` resolves the caller’s future. The type system gives each method a return type, and every `reply` in the body together with every `now` / `future` call site shares that type.

- `reply` happens **at most once**. A second one would arrive after the waiter already took the first value, so it would be silently dropped. That is made a type error.
- If the return type is *declared* as something other than `unit`, the checker verifies that **every execution path replies exactly once**. The body of a `while` may run zero times, so it is conservatively judged as “there is a path that does not reply”.
- A method with no `reply` at all is treated as returning `unit`. Notification-style methods called with `send` are of this kind.

```
method m(k) : int { reply(k); reply(k + 1); }           // type error
method m(k) : int { if (k > 0) { reply(1); } else { reply(2); } } // OK
```

9.3 select

```
select {  
  case job(k) -> { reply("done:" ++ k); }  
  timeout 500 -> { print("timed out"); }  
}
```

`case m(x1, ..., xn)` is matched against the signature of the method of the same name. A mismatch in the number or type of arguments is a type error. `timeout` is in milliseconds and may be omitted.

A reply in a case body answers “the selected message”, not the enclosing method. The implementation switches the reply destination to the selected message before running the case, so the `reply` above answers `job`. Hence a method containing a `select` may itself be `unit`. The `timeout` body, by contrast, runs with the enclosing method’s reply destination.

`select` looks for messages that have *not yet been processed*. If ordinary method dispatch takes the message of that name first, nothing is left for the case, so it is safer to design the names you `select` on so that nobody calls them directly (see the output in §26).

10 Waiting with a deadline

10.1 Why it is needed

A `now` or `await` without a deadline waits forever if no reply comes. An actor is one thread and a wait blocks on a condition variable, so while it waits it cannot process any other message. It turns to stone.

```
now t.m(a) timeout <milliseconds> else <expression>  
await f    timeout <milliseconds> else <expression>
```

If the reply arrives in time you get its value; otherwise you get the value of the `else` branch.

10.2 The rules

- The `else` branch must have **the same type as the body**, because a timeout must not change the type of the whole expression. Otherwise: `now:` the `else` branch has type `string` but the call returns `int`.
- The deadline must be **positive** (timeout must be positive).
- A `now` / `await` without a deadline is a **warning** by default, and an **error** under `AIOS.STRICT_DEADLINE=1`.
- The upper bound on the number of replies is the `max` of the body and the `else` branch (not their sum), since only *one* of them runs.
- A rejected future (`reject`) is distinguished from a timeout and raises an exception.

In the implementation, OCaml’s `Condition` has no timed wait, so the deadline is measured by polling at 1 ms granularity.

10.3 Relation to type soundness

Writing a deadline makes every wait finish in finite time. But that only means *a deadlock is converted into a failure in finite time*; the failure does not disappear. Two actors waiting on each other with `now` both time out. Nothing is stuck — and nothing is right either.

To say **that nothing gets stuck** in the type system you need not deadlines but **obligation levels** (§15). Those are **machine-checked in Coq/Rocq**: for the core *including* `await`, type

soundness, type safety and deadlock freedom are proved with no axioms (`coq/AIPLSoundness2.v`). The first version could state deadlock freedom only for a fragment with `await` removed, which is what the second version fixes.

The progress theorem changed as follows.

What progress states	
1st version	terminal state $\vee$ can step $\vee$ all tasks are waiting
2nd version	terminal state $\vee$ can step

Termination is still not claimed. `while 1 > 0 do {}` does not stop, but it is not a wait, so it does not contradict progress.

11 Effects

11.1 The eight effects

Effect	Meaning	Effect	Meaning
<code>mut</code>	assignment to a field	<code>net</code>	communication outside the node
<code>time</code>	reading the clock, waiting	<code>ai</code>	model inference
<code>io</code>	input/output to a device	<code>fs</code>	persistence
<code>mem</code>	dynamic allocation	<code>log</code>	output for observation only

This is not a newly designed taxonomy: it is the capability classification the evaluator already carried as run-time metadata, made static. The point is that `ai` and `net` are separate — a model call that leaves the box has both, while on-device inference gets `{ai}` only. “Uses AI but never leaves the box” becomes visible in the signature.

11.2 How to write them

```
method plan(goal) : string ![ai, net] { ... } // after the return type
method peek()           !{}      { ... } // explicitly effect-free
method tick()           { ... } // omitted: inferred
```

The annotation is optional (gradual). If you omit it, inference simply runs and nothing is said. If you write it, “the effects of the body $\subseteq$ the effects declared” is checked. Declaring more than you use is allowed. An unknown effect name is rejected as a typo.

```
[Type error] line 1, col 18: unknown effect(s) network in A.m;
known effects are mut, time, io, mem, net, ai, fs, log
```

11.3 What counts as an effect

Construct	Effect
assignment to a field	<code>{mut}</code>
assignment to a local	not an effect
<code>new C(...)</code>	<code>{mem}</code>
calling a builtin	that function’s effects (§22)
send to a <code>remote(...)</code> target	<code>{net}</code>
local send with <code>now</code>	inherits the callee’s effects
<code>send / future</code>	does not inherit

That `mut` covers only assignment to fields is deliberate: it lets a pure actor (safe to replicate) and an actor with external effects (exactly one at a time) be told apart by the effect set alone.

Propagation through `now` iterates to a fixpoint. Effects only grow, so it terminates.

11.4 What this lets you forbid

```
class Controller {
  method step(s) : string !{io, time} {
    var r = now planner.plan(s) timeout 100 else ""; // plan is !{ai, net}
    reply(r);
  }
}
```

```
[Type error] method Controller.step declares {io, time}
but its body has {ai, net, io, time} (missing {ai, net})
```

“A real-time actor synchronously calls an agent outside the box” is forbidden by the effect check alone. What used to be stated as a design decision now holds mechanically in the type system.

A known hole. Splitting a `now` into `future` and `await` *escapes* this check: the implementation adds the propagation edge only at the `now` site and not at the `await`. A reproduction is in `docs/samples/soundness2_effect_gap.aipl`. The fix (carrying effects in the `future` type) is described in the second soundness report.

Part III

The seven disciplines

On top of types, effects and deadlines sit seven disciplines. All of them are built so that they **work without annotations** — because the information they need is usually already written in the program.

Discipline	How you write it	What it stops
Effects	<code>!{ai, net}</code>	the real-time side waiting on a model outside the box
Deadlines	<code>timeout n else e</code>	waits that never return
Failure type	<code>timeout n</code> (no <code>else</code>)	letting a timed-out value flow on as a normal one
First-class reply	<code>replyto</code> / <code>answer</code> / parameter type <code>reply</code>	dropped replies and double replies
Resource order	<code>acquire</code> / <code>release</code> / <code>resource_order</code>	forgetting to release, double acquire, acquiring in reverse order
Obligation levels	<code>@n</code> (inferred if omitted)	cycles of waiting
Session types	<code>protocol_define</code> / <code>_start</code> / <code>_end</code>	steps taken out of order

12 Putting failure in the type — `result< $\tau$ >`

If you write `else` on a deadline, the timed-out value and the normal value have **the same type**. The program cannot ask whether `-1` came back because it timed out or because `-1` was computed.

Write no `else` and you get `result< $\tau$ >`.

How you write it	Type
<code>now a.m(x) timeout 200 else -1</code>	<code>int</code>
<code>now a.m(x) timeout 200</code>	<code>result<int></code>

Use `is_ok` / `timed_out` / `value` to take the value out.

```
var a = now s.fast(42) timeout 200;      // result<int>
if (is_ok(a)) { print("fast ok " ++ value(a, -1)); }
else          { print("fast timed out"); }
```

A `result $\langle\tau\rangle$` cannot be used as a number. Feeding it into arithmetic without checking stops the program at compile time.

```
no overload of + matches (result int, int)
```

13 Making the reply destination a first-class value

`replyto` yields, as a value, the reply destination of the message currently being processed. Its type is `reply $\langle\rho\rangle$` (where ρ is the method’s return type), and `answer(r, v)` sends the reply. `reply(v)` is sugar for that.

With the parameter annotation `reply` you can hand it to another actor. A front desk takes the request and a specialist answers **directly**.

```
class Backend {
  method handle(r: reply, x) : unit !{} {
    answer(r, x * 2);           // answer Frontend's caller directly
  }
}
class Frontend {
  var b = new Backend();
  method ask(x) : int !{} {
    send b.handle(replyto, x); // pass the destination on; do not reply here
  }
}
```

A reply destination is **linear**: using it exactly once is an obligation.

Rule	Content
Creation	<code>var r = replyto</code> ; creates the obligation
Consumption	<code>answer(r, v)</code> , or passing it as a send argument, discharges it
Exactly once	the same <code>r</code> must not be used twice
Nothing left	no obligation may remain when the method returns
Branches	the two branches of an <code>if</code> must join in the same state

The check is closed within a method. A reply destination **cannot be kept in a field** (a field’s type is fixed by its initializer). A bounded buffer that “collects senders and answers them later” therefore cannot be written directly in the core as it stands.

14 Resource order

Effects are a **set**, so they cannot express order; “take it and give it back” cannot be said. So the `acquire` / `release` pairs are tracked through the body instead.

```
class Bus {
  var n = 0;
  method read(addr) : int !{mut} {
    acquire("i2c");
    n = n + 1;
    var v = addr * 2;
```

```

    release("i2c");           // delete this line and it is a type error
    reply(v);
  }
}

```

There are three rules. Nothing may still be held when the method returns. You may not release what you do not hold. You may not acquire twice. The two branches of an `if` must join holding the same things, and the body of a `while` must not change what is held.

14.1 A global order — stopping reverse acquisition

Checking pairs is not enough. If two actors take the same two resources in **opposite order**, both pairings are correct, yet at run time they wait for each other.

No new annotation is needed. The nesting “*s* was acquired while *r* was held” *is* the claim $r < s$. The claims are collected from the whole program and checked for cycles.

```

resource order: cache -> db -> cache is circular;
  cache before db (in Reader.run); db before cache (in Writer.run).
  Acquiring in opposite orders deadlocks

```

Every edge of the cycle carries **where it came from**, so the two places that acquire in opposite order are named directly.

You may also write `resource_order("bus -> dev")` instead of relying on inference. A declaration simply enters the same table as an edge, so if the declaration and the actual acquisitions disagree, that shows up as a cycle too. The inferred order can be printed with `AIOS_SHOW_LEVELS=1`.

```

[resource] bus           @0
[resource] dev           @1

```

Only `acquire` calls whose resource name is a **string literal** can be tracked. A name held in a variable is invisible statically, so the declared order is also enforced at run time (`AIOS_STRICT_RESOURCE=1` reports the places that could not be tracked).

15 Obligation levels — stopping cycles of waiting

There are four shapes in which a wait never returns: a cycle of waits, a wait that is never replied to, a `select` waiting for a message that never comes, and resources acquired in reverse order. **Obligation levels** address the first.

Each method gets a number, and **a wait must always go to a strictly larger number**.

```

class Leaf { method f(x) : int @2 !{} { reply(x + 1); } }
class Mid  { var l = new Leaf();
  method g(x) : int @1 !{} { reply(now l.f(x) timeout 200 else 0); } }

```

You do not have to write them. If you omit them, they are inferred by a fixpoint that pushes levels up along the wait edges. An explicit annotation is treated as a fixed point; if it cannot be satisfied, that is where the contradiction is.

```

obligation level: B9#g (@2) waits on A9#f (@2);
                  a wait must go to a strictly higher level
obligation levels do not converge; the wait graph has a cycle

```

The inferred levels can be printed with `AIOS_SHOW_LEVELS=1`.

15.1 Waits that cross nodes

In a mesh the target lives on another node and is invisible statically. So each node declares a **floor** for its levels, and a wait towards that node must go up.

```
node_level("rt0", 10);           // the real-time node is a leaf (high level)
```

```
obligation level: Ctl2#go (@7) waits on node rt0 (floor @1);
                a wait across nodes must go up
```

This is exactly the same shape as checking effects at the border with `node_allow`.

15.2 How much is guaranteed

As long as the target of a wait is known as a class, or the level of the target node is declared, deadlock freedom is guaranteed by the type system. This is not a claim on paper only — it is machine-checked in Coq/Rocq for the core *including* `await` (`coq/AIPLSoundness2.v`, no axioms).

The conditions matter. A dynamic target is invisible. Levels are per method, so two instances of the same class are treated alike. Termination and starvation are separate problems.

16 Session types — the order of an exchange

“Open it, then read, then close it at the end” is a promise that neither deadlines nor the existence of a sender can express. AIPL already had **run-time** session protocols, so the type checker reads **the same declaration**. No new declaration is needed.

```
protocol_define("Pipeline", "f.get -> r.scale -> st.put");
var sid = protocol_start("Pipeline");

var a = now f.get(4)      timeout 200 else 0;
var b = now r.scale(a)    timeout 200 else 0;
var c = now st.put(b)     timeout 200 else 0;
protocol_end(sid);
```

Get the order wrong and it is reported before the program runs.

```
protocol Pipeline: expected r.scale but the program sends st.put here
protocol Pipeline is incomplete at protocol_end; next expected st.put
```

(An example of real output.)

```
protocol P29: expected b.step2 but the program sends c.step3 here
```

16.1 When the session crosses actors

In a real program the steps do not fit in one straight sequence: you call one front-desk actor, and the remaining steps happen inside its body.

A synchronous send (`now` / `aios_now`) **runs to completion before** the caller continues. So the callee’s sequence of sends can be spliced in at the call site (behaviour expansion), and a session that crosses actors can still be matched as a single sequence.

Asynchronous sends (`send` / `future`) are not spliced in — there is no telling when they run. If the callee contains steps, completion is left to the run-time check (`AIOS_STRICT_PROTOCOL=1` reports it).

Steps refer to roles **by name**, so **pass actors as arguments**. If you keep them in fields, the run-time name (`c#f`) and the static name (`f`) disagree and the step cannot point at the role.

17 The select discipline

Two things are required of a `select`.

1. **Write a deadline.** A `select` without a `timeout` clause waits forever for a message that may never come.
2. **Check that a sender exists.** If nobody in this program sends the `m` of `case m(...)`, it is either a typo or something you forgot to write.

```
select waits for W2.ghost but nothing in this program sends it
select without a timeout waits forever; write 'timeout <ms> -> { ... }'
```

Both are warnings by default. When sends are dynamic, or the program is exposed to the outside, no judgement is possible, so the check is simply not made — **the language does not see the whole world**.

18 Mesh deployment

An actor's source is shipped to another node and is parsed and type-checked **at the destination** before it is instantiated. The effect annotations travel with the code and are matched against the receiving node's policy.

```
node_allow("rt0", "mut, log, time");    // effects the real-time node accepts
deploy("rt0", "Servo", "servo1");      // ship Servo's source to rt0 and instantiate
var d = now remote("rt0", "servo1").step(3) timeout 200 else -1;
```

```
[deploy] Servo -> rt0/servo1 (compiled at destination)
```

Ship code carrying `ai` or `net` to a real-time node and it is stopped at the border. Effects turn from a discipline inside one node into **immigration control between nodes**.

19 Escape hatches (environment variables)

Every default is on the strict side. Environment variables exist that loosen things, but they are not meant for everyday use. The list is in §40.

Part IV

Reference

20 What the type checker looks at

Check	Content
Method existence	does the local target have a method of that name
Argument count and type	do the actual arguments match what the body requires
reply type	do all <code>reply</code> calls have the same (declared) type
reply count	at most once; with an annotation, exactly once on every path
select matching	do the <code>case</code> arity and argument types match the signature
Annotations when exposed	do the methods of a <code>web.exposed</code> actor carry annotations
Overloads	is the candidate unique (ambiguity is an error)
Effects	do the declared effects cover the body's; are the effect names known
Deadlines	the type of the <code>else</code> branch, positivity, presence
Failure type	is a <code>result(τ)</code> used without being checked
Reply linearity	is <code>replyto</code> used exactly once
reply totality	does a method waited on by <code>now/await</code> reply on every path
Resources	the <code>acquire/release</code> pairs and cycles in the acquisition order
Obligation levels	does every wait go up; the same across node borders
Sessions	is the order of sends the order of <code>protocol.define</code>
select	is there a deadline; does anybody send the awaited message
Assignment to undeclared names	is a name assigned to without a <code>var</code> declaration
Deployment border	do the shipped code's effects fit the receiver's policy

Representative messages:

```
no method foo in actor(C)
type mismatch                                <- argument or assignment type
no overload of + matches (string, int)       <- + is numeric only
ambiguous overload of + for ('a, 'a): candidate return types are float / int
reply type mismatch: this method is declared to reply with int,
  but replies with string here
method m is declared to reply with int, but some execution path does not reply
method m may reply more than once on some path
select: case m binds 1 variable(s) but method m takes 2
actor C is reachable from outside this program, but m has no return type
  annotation; ...
method C.m declares {log, time} but its body has {ai, log, net}
  (missing {ai, net})
unknown effect(s) network in A.m; known effects are mut, time, io, mem, ...
now: the else branch has type string but the call returns int
now: timeout must be positive
now without a deadline waits forever;
  write 'now ... timeout <ms> else <expr>'      <- warning by default
assignment to undeclared name: cout (write 'var cout = ...' to declare it)
no overload of + matches (result int, int)     <- check a result before using it
method Silent4.f is waited on by now/await but does not reply on every path
resource i2c is released without being acquired
method Dev4.use returns while still holding i2c
resource order: cache -> db -> cache is circular; ...
obligation level: B9#g (@2) waits on A9#f (@2);
  a wait must go to a strictly higher level
obligation levels do not converge; the wait graph has a cycle
protocol P29: expected b.step2 but the program sends c.step3 here
select waits for W2.ghost but nothing in this program sends it    <- warning
select without a timeout waits forever                                <- warning
```

```
deploy: Servo.step is @3 but node rt0 has floor @10
```

21 Exposing to the outside

`web_listen(port)` starts an HTTP gateway, and `web_expose(path, actorName)` gives an exposed endpoint an alias.

Endpoint	Content
GET /	a small UI
POST /api/send	to=<actor>&method=<m>&args=<a,b> reaches any actor
POST /api/x/<key>	reaches the alias registered with <code>web_expose</code>

POST /api/send can reach any actor by name. In other words it is `web_listen` that opens the border; `web_expose` only adds an alias. The type checker takes that into account: for an actor named by `web_expose`, a return annotation on every method (except `init`) is **mandatory**, and with `web_listen` alone it is a **warning**. The far side cannot see the body, so the reply type can only travel as a declaration.

`AIOS_LAX_EXPOSE=1` downgrades the error to a warning.

Arguments arriving over HTTP are built by interpreting the string as `int` $\rightarrow$ `float` $\rightarrow$ `string`, in that order. An annotation cannot make the boundary less dynamic, but it can declare *which type you promise*.

22 Builtin functions and their effects

Group	Names	Effect
Math	<code>sin</code> , <code>cos</code> , <code>tan</code> , <code>asin</code> , <code>acos</code> , <code>atan</code> , <code>sqrt</code> , <code>exp</code> , <code>log10</code> , <code>abs</code> , <code>floor</code> , <code>ceil</code> , <code>round</code>	{}
Basic	<code>typeof</code> , <code>reply</code> , <code>neg</code>	{}
Output	<code>print</code>	{log}
Time	<code>wait</code>	{time}
Arrays (read)	<code>array_get</code> , <code>array_len</code>	{}
Arrays (allocate)	<code>array_empty</code> , <code>array_push</code> , <code>array_set</code>	{mem}
Actor creation	<code>spawn</code>	{mem}
Drawing (SDL)	<code>sdl_init</code> , <code>sdl_clear</code> , <code>sdl_line</code> , <code>sdl_erase_line</code> , <code>sdl_present</code> , <code>sdl_line_c</code>	{io}
Memory and tasks	<code>aio_memory_put/get/has/keys</code> , <code>aio_task_create/set/get/info</code> , <code>aio_tasks</code>	{fs}
Model inference	<code>ai_call</code> , <code>ai_call_with_system</code> , <code>model_generate</code> , <code>gemini_generate</code> , <code>openai_generate</code>	{ai, net}
Communication and exposure	<code>web_listen</code> , <code>web_expose</code> , <code>aio_now</code> , <code>aio_future</code> , <code>remote_review</code> , <code>remote_review_ja</code> , <code>remote_reviewer_host</code>	{net}
Introspection	<code>capabilities</code> , <code>capability_prims</code> , <code>aio_kernel</code> , <code>aio_actors</code> , <code>aio_actor_info</code> , <code>aio_actor_methods</code> , <code>aio_mailbox_len</code> , <code>actor_dump</code>	{log}
Events	<code>aio_emit</code> , <code>aio_events</code> , <code>aio_events_since</code> , <code>aio_event_count</code>	{log}
Protocols	<code>protocol_define/start/use/current/state/end/events</code>	{log}
Services	<code>aio_register_service</code> , <code>aio_services</code> , <code>aio_service_actor</code> , <code>aio_service_info</code>	{log}

A primitive that is not in this table is treated as effect-free.

Part V

Sample programs

The samples in this part are the ten programs in `docs/samples/guide/`. All of them type-check, their output is exactly as printed here, and **the three implementations (OCaml, Python, JavaScript) agree on that output** — verified with `tools/run_all_impls.sh`. (The sources in the repository carry Japanese comments; the listings below are the same programs with the comments translated.)

Samples for the more advanced features (`result< $\tau$ >`, delegating a reply destination, resource order, obligation levels, session types, mesh deployment) are the twenty programs in `docs/samples/paper/`; the examples in Part III come from there.

23 g1: classes, fields, send, string concatenation

```
// g1: class, field, init, method, send, string concatenation
class Greeter {
  var count = 0;           // a field, initialized as int

  method init(name) {      // called automatically by new
    print("hello, " ++ name); // ++ concatenates (both sides are stringified)
  }

  method tick() : unit {   // return annotation; no reply, so unit
    count = count + 1;     // + is numeric only
    print("tick " ++ count);
  }
}

var g = new Greeter("AIPL");
send g.tick();             // asynchronous; does not wait for a reply
send g.tick();
```

```
hello, AIPL
tick 1
tick 2
```

`init` is called automatically by `new Greeter("AIPL")`. `tick` does not `reply`, so it declares `unit` and the coverage check does not apply. `send` does not wait for a reply, so the two `ticks` are processed in order. `count + 1` is `int` with `int`, so it stays `int` and prints as 1, 2.

The inferred effects are `Greeter#init !{log}` and `Greeter#tick !{log, mut}`.

24 g2: now / future / await and annotations

```
// g2: now / future / await and the return annotation
class Calc {
  // with an annotation, every reply is matched against this type and
  // replying on every path is checked as well
  method twice(a) : int {
    reply(a * 2);
  }

  // without one, the type is inferred from reply (-> string)
  method label(a) {
    reply("v=" ++ a);
  }
}
```

```

    }
}

var c = new Calc();

var s = now c.twice(21);      // now is an expression; the result is int
print("twice = " ++ (s + 1)); // usable as int in arithmetic

var f = future c.twice(10);   // future gives future int
var v = await f;              // await takes the value out
print("awaited = " ++ v);

print(now c.label(7));        // the inferred string

```

```

twice = 43
awaited = 20
v=7

```

`twice` declares `int`, so `now c.twice(21)` is `int` and can be used in the arithmetic `s + 1`. `label` omits the annotation, and `string` is inferred from `reply("v=" ++ a)`.

This sample writes no deadlines, so the type checker emits three warnings.

```

[warn] line 16, col 9: now without a deadline waits forever;
      write 'now ... timeout <ms> else <expr>'

```

25 g3: several actors, and a now across actors

```

// g3: several actors, and a now that crosses actors
class Stock {
  var n = 10;

  method take(k) : int {
    n = n - k;
    if (n < 0) { n = 0; }
    reply(n);
  }
}

class Front {
  // Stock.take returns int; that becomes the type of the now,
  // which in turn feeds place's own string
  method place(k) : string {
    var left = now stock.take(k);
    if (left > 0) {
      reply("ok, left=" ++ left);
    } else {
      reply("sold out");
    }
  }
}

var stock = new Stock();
var front = new Front();

print(now front.place(3));
print(now front.place(99));

```

```
ok, left=7
sold out
```

`Stock.take` returns `int`; that becomes the type of the `now` inside `Front.place`, which then feeds `place`'s own `string`. Types propagate across actor boundaries.

Because `place` declares `string`, the checker verifies that both branches of the `if` reply. Delete one and you get some `execution path does not reply`.

Note that `n = n - k` with `int n` forces `k` to be `int`, consistent with `place(3)` at the call site; writing `place("x")` would be an *argument* type error.

Both methods have effect `{mut}` (assignment to field `n`), and `Front#place` inherits `Stock#take`'s `{mut}` through the `now`.

26 g4: select, and which message a reply answers

```
// g4: select / case / timeout, and which message a case's reply answers
class Worker {
  // the job message that arrives from outside
  method job(k) : string {
    reply("direct:" ++ k);
  }

  // wait for job with select. The reply in the case body answers job,
  // so serve itself replies to nothing and stays unit
  method serve() : unit {
    print("waiting");
    select {
      case job(k) -> {
        print("got " ++ k);
        reply("via select:" ++ k);
      }
      timeout 500 -> {
        print("timed out");
      }
    }
  }
}

var w = new Worker();
send w.serve();
print(now w.job(1));
```

```
direct:1
waiting
timed out
```

The `reply` in the body of `case job(k)` answers `job`, not `serve`, so `serve` may stay `unit`. The type checker also matches that `reply` against `job`'s return type.

The output shows **a design caveat this example carries**. Ordinary method dispatch processed `now w.job(1)` first, so by the time `serve`'s `select` ran, no `job` was left in the mailbox and it fell through to the `timeout`. It is safer to design the message names you `select` on so that they are not called directly.

27 g5: exposing to the outside makes annotations mandatory

```
// g5: exposing an actor makes the return annotation mandatory
//      web_listen opens the border, and POST /api/send can reach any actor.
//      The far side cannot see the body, so the reply type can only
//      travel as a declaration.
class Echo {
  method say(msg) : string { // without the annotation this is a type error
    reply("echo: " ++ msg);
  }
  method note(msg) : unit {
    print("[note] " ++ msg);
  }
}

var echo = new Echo();

web_listen(8080);
web_expose("/echo", "echo");

print("try POST /api/x/echo with method=say&args=hi");
```

This program type-checks. Remove the annotation on `say` and you get:

```
[Type error] line 16, col 1: actor Echo is reachable from outside this
program, but say has no return type annotation; a remote caller cannot
see the body, so the reply type has to be declared (method say(...) : T)
```

Running `web_listen(8080)` starts the gateway and blocks listening, so check it from another terminal.

```
$ curl -s -X POST http://localhost:8080/api/x/echo \
  -d 'method=say&args=hi'
```

28 g6: effect annotations and propagation

```
// g6: the effect annotation !{...}
//
//      The existing capability classification, made static. Eight effects:
//      mut write your own state      net communication outside the node
//      time reading the clock, waiting ai model inference
//      io input/output to a device    fs persistence
//      mem dynamic allocation         log output for observation only
//
//      Omit the annotation and it is inferred from the body. Write it and
//      "the body's effects the declared effects" is checked (more is allowed).
class Counter {
  var n = 0;

  method bump() : int !{mut} { // assignment to a field is mut
    n = n + 1;
    reply(n);
  }
  method peek() : int !{} { // effect-free: pure, so free to replicate
    reply(n);
  }
  method show() : unit !{log} { // print is log
    print("n = " ++ n);
  }
}
```

```

class Planner {
  // ai_call leaves the box, so it has both ai and net.
  // It shows up in the signature, so which actors leave the box is visible
  method plan(goal) : string !{ai, net} {
    reply(ai_call("plan: " ++ goal));
  }
}

class ViaNow {
  // now waits, so it inherits the callee's effects.
  // Without declaring {ai, net} this is a type error
  method run(g) : string !{ai, net} {
    var r = now planner.plan(g);
    reply(r);
  }
}

class ViaSend {
  // send does not wait, so it does not inherit. Effect-free is fine
  method fire(g) : unit !{} {
    send planner.plan(g);
  }
}

var planner = new Planner();
var c = new Counter();
var a = new ViaNow();
var b = new ViaSend();

// ai_call would hit an external API, so only Counter is exercised here
print(now c.bump());
print(now c.peak());
send c.show();

```

```

1
1
n = 1

```

The effect table printed by the type checker is the point of this sample.

```

[method effects (inferred / declared)]
Counter#bump !{mut}      <- assignment to a field
Counter#peak !{}        <- pure
Counter#show !{log}     <- print
Planner#plan !{ai, net} <- ai_call
ViaNow#run  !{ai, net}  <- now, so it inherited plan's effects
ViaSend#fire !{}        <- send, so it did not

```

Remove the {ai, net} declaration from ViaNow.run and it fails with declares {} but its body has {ai, net}. ViaSend.fire may stay {}. The rule that **only the side that waits carries the callee's effects** is visible right here.

29 g7: now / await with deadlines

```

// g7: now / await with a deadline
//
//   Without a deadline, a now or await waits forever if no reply comes.
//   An actor is one thread, so while it waits it cannot process any

```

```

//      other message -- it turns to stone.
//
//      now t.m(a) timeout <ms> else <expr>
//      await f      timeout <ms> else <expr>
//
//      The else branch must have the same type as the body.
class Slow {
  method work(k) : int !{time} {
    wait(300);           // takes 300 ms
    reply(k * 2);
  }
  method fast(k) : int !{} {
    reply(k * 2);
  }
}

var s = new Slow();

// returns in time
var a = now s.fast(21) timeout 1000 else 0;
print("fast = " ++ a);

// times out -> the value of the else branch
var b = now s.work(21) timeout 50 else 0;
print("slow(timed out) = " ++ b);

// awaiting a future with a deadline
var f = future s.fast(5);
var c = await f timeout 1000 else 0;
print("await = " ++ c);

```

```

fast = 42
slow(timed out) = 0
await = 10

```

`work` takes 300 ms, so the `now` with a 50 ms deadline times out and becomes the 0 of the `else` branch. `fast` makes the 1000 ms deadline and returns 42.

This sample also passes under `AIOS_STRICT_DEADLINE=1` (every wait has a deadline). Change `else 0` to `else "zero"` and you get:

```
[Type error] now: the else branch has type string but the call returns int
```

30 g8: equality, unary minus, boolean literals

```

// g8: equality, unary minus, boolean literals
//
//      The lexer and the evaluator had all of these, but the expression
//      grammar did not, so they could not be written.
//
//      ==  !=          equality, on int / float / string / bool
//      - e          unary minus; lowers to neg(e)
//      true false    boolean literals
//
//      Precedence is equality < comparison < ++ < additive < multiplicative
//      < unary minus, so 1 + 2 == 3 reads as (1 + 2) == 3.
class C {
  var flag = false;           // a bool field
  method set(v) : unit !{mut} { flag = v; }
}

```



```

method get() : bool !{} { reply(flag); }
method t() : unit !{log, mut} {
  flag = true;
  if (flag == true) { print("literal true ok"); }
  if (flag != false) { print("literal false ok"); }
  var b = 1 < 2;
  print("b = " ++ b);
  print("not-eq: " ++ (true != false));
}
}
var c = new C();
send c.t();

```

```

literal true ok
literal false ok
b = true
not-eq: true

```

All of these existed in the lexer and the evaluator but had *no expression grammar rule*, so they could not be written; the previous edition listed them as a known gap. `flag` is initialized with `false`, so it is `bool`, and the parameter of `set(v)` is wired to `bool` as well.

31 g9: passing an actor as an argument

```

// g9: passing an actor as a message argument
//
//      With the parameter annotation 'method m(x: D)' you can send to the
//      actor you received. Without it the parameter type is undetermined
//      and you get 'send target is not actor'.
class D { method ping() : unit !{log} { print("pong"); } }
class P {
  method viaAnno(x: D) : unit !{log} {
    print("got actor");
    send x.ping();
    print("after send");
  }
}
var d = new D();
var p = new P();
send p.viaAnno(d);

```

```

got actor
after send
pong

```

The parameter annotation `d: Device` fixes the class of that argument; passing anything else is a type error. This annotation also adds an edge for obligation levels (§15), which is exactly why levels did not need to be carried on actor types.

32 g10: two classes with a field of the same name

```

// g10: two different classes may have a field of the same name
//
//      A class body is checked under that class's own fields, so
//      Fork's held and Latch's held are different things.
class Fork {

```

```

var held = 0;
method take() : int !{mut} { held = 1; reply(held); }
}
class Latch {
  var held = 0;
  method arm() : int !{mut} { held = 2; reply(held); }
}
var f = new Fork();
var l = new Latch();
print("fork = " ++ (now f.take() timeout 200 else -1));
print("latch = " ++ (now l.arm() timeout 200 else -1));

```

```

fork = 1
latch = 2

```

This is a regression test. The OCaml implementation used to *stack* fields into the type environment, so an assignment to the second class's `held` was rejected as an overloaded name (Python and JavaScript were correct). Of the 586 programs at hand, 119 contained two classes with a field of the same name, but in almost all of them another error came first and hid it.

33 Negative samples

`docs/samples/reply_inference/` holds samples whose purpose is to confirm that they *do not* pass.

File	What happens
<code>s2_missing_reply</code>	the result of a forgotten <code>reply</code> is used in arithmetic
<code>s3_conflict</code>	branches <code>reply</code> with different types
<code>s5_overload_ambiguity</code>	<code>a + b</code> with no principal type
<code>s8_annotation_conflict</code>	a <code>reply</code> that contradicts the declaration
<code>s9_missing_path</code>	some path does not <code>reply</code>
<code>s10_expose_unannotated</code>	an exposed actor without annotations
<code>s12_service_exposed</code>	the same, in the exposed integration sample
<code>s14_select_pattern</code>	a <code>case</code> arity that differs from the signature
<code>s15_double_reply</code>	a double <code>reply</code>
directly under <code>docs/samples/</code>	
<code>soundness2_effect_gap</code>	<code>future+await</code> escapes the effect check (a known hole)

Part VI

Building and running

34 Building the implementation

```
cd src && make thread_repl
```

Building with `dune` fails because `src/lexer.ml` is generated twice, so use `src/Makefile`.

35 Running a program

Pass a file of REPL commands with `-f`.

```

cat > run.repl <<'EOF'
load docs/samples/guide/g7_deadline.aipl
compile

```

```
EOF
./src/abclrepl_thread -q -f run.repl < /dev/null
```

The main REPL commands are `load` / `compile` / `list` / `send` / `ast` / `pprint` / `script` / `exit`.

36 Running only the type checker

The REPL depends on SDL, so a small driver exists for type checking alone.

```
ocamlfind ocamlc -package unix -thread -linkpkg -w -a -I src -o tc \
  src/location.cmo src/types.cmo src/ast.cmo src/typing_env.cmo \
  src/infer.cmo src/typecheck.cmo src/parser.cmo src/lexer.cmo \
  docs/samples/reply_inference/tc_main.ml

./tc docs/samples/guide/g3_actors.aipl
```

It prints the table of inferred and declared return types, the effects of each method, and the signature of each class.

37 The type/run-time differential test

There is a harness that compares the return type the checker assigned with the type of the value actually `replied` at run time.

```
python3 scripts/type_runtime_diff.py abclc/*.aipl docs/samples/guide/*.aipl
```

With `AIOS_TYPE.TRACE=1` the implementation prints `[rtype] Class#method = tag` on every `reply`. This mechanism actually found an inconsistency where integer arithmetic returned a `float` ($\vdash e : \text{int}$ yet $e \Downarrow \text{VFloat}$). A deliberately mismatching negative sample can be marked as known with `// TYPE_RUNTIME_DIFF: expect-mismatch`.

Run it whenever you change the types. It is the standing device for catching a break in preservation.

38 Environment variables

Variable	Effect
<code>AIOS_QUIET=1</code>	suppress debug output such as type schemes
<code>AIOS_STRICT_DEADLINE=1</code>	make a <code>now</code> / <code>await</code> without a deadline an error
<code>AIOS_LAX_OVERLOAD=1</code>	make an ambiguous overload a warning instead of an error
<code>AIOS_LAX_EXPOSE=1</code>	make a missing annotation on an exposed actor a warning
<code>AIOS_TYPE_TRACE=1</code>	print an <code>[rtype]</code> line on every <code>reply</code>
<code>AIOS_MODEL_PROVIDER</code>	choose the provider for AI calls
<code>ABCL_AI_PROVIDER</code>	the same (old name)
<code>REMOTE_REVIEWER_HOSTPORT</code>	the target of <code>remote_review</code>

Part VII

Appendix

39 Grammar summary

```
program    ::= decl+
decl       ::= "class" ID "{" field* method+ "}"
```

```

    | "var" ID "=" expr ";"
    | "var" ID "=" "new" ID "(" args ")" ";"
    | "send" target "." ID "(" args ")" ";"
    | "send!" target "." ID "(" args ")" ";"
    | ID "(" args ")" ";"

field      ::= "var" ID "=" expr ";"
           | "float" ID "=" expr ";"

method     ::= "method" ID "(" params ")" ret? lvl? eff? "{" stmt+ "}"
params     ::= (param ("," param)*)?
param      ::= ID | ID ":" ClassName | ID ":" "reply"
ret        ::= ":" ("int"|"float"|"string"|"bool"|"unit"|"any"|ClassName)
lvl        ::= "@" INT -- obligation level; inferred if omitted
eff        ::= "!" "{" (ID ("," ID)*)? "}"

target     ::= ID
           | "remote" "(" STRING "," STRING ")"

stmt       ::= ID "=" expr ";"
           | "var" ID "=" expr ";"
           | "var" ID "=" "new" ID "(" args ")" ";"
           | ID "(" args ")" ";"
           | "call" ID "(" args ")" ";"
           | "send" ("self"|"sender"|target) "." ID "(" args ")" ";"
           | "send!" target "." ID "(" args ")" ";"
           | "if" "(" expr ")" stmt ("else" stmt)?
           | "while" expr "do" stmt
           | "{" stmt* "}"
           | "select" "{" case* timeout_case? "}"

case       ::= "case" ID "(" ids? ")" "->" "{" stmt+ "}"
timeout_case ::= "timeout" INT "->" "{" stmt+ "}"

expr       ::= INT | FLOAT | STRING | "true" | "false" | ID
           | "-" expr
           | expr ("=="|"!="|">"|"<"|">="|"<="|"++"|"++"|"-+"|"*+"|"/") expr
           | "new" ID "(" args ")"
           | "now" target "." ID "(" args ")" deadline?
           | "future" target "." ID "(" args ")"
           | "await" expr deadline?
           | "replyto" -- the reply destination as a value
           | ID "(" args ")"
           | "(" expr ")"

-- with else it is tau; without else it is result<tau>
deadline   ::= "timeout" INT "else" expr
           | "timeout" INT

```

Operator precedence, weakest first: `== != < > < >= <= < ++ < + - < * / <` unary minus. Every binary operator is **left associative** (equality included) — `1 == 2 == false` reads as `((1 == 2) == false)` and evaluates to `true`. Unary minus is a prefix and can be stacked (`- - 3` is `3`).

40 Environment variables of the implementation

Some variables change how checking and execution behave. Set them to 1 to enable; the default is always “not enabled”.

Variable	Effect
<code>AIOS_STRICT_DEADLINE</code>	make a now / await without a deadline an error ; by default it is only a warning. Set it if you want to stay inside the fragment for which the soundness theorem holds (AIPL ⁻²)
<code>AIOS_LAX_OVERLOAD</code>	restore the old behaviour for ambiguous overloads: warn and pick the default candidate. The default is strict (an error)
<code>AIOS_LAX_EXPOSE</code>	downgrade a missing annotation on an exposed actor (§21) to a warning. The default is an error
<code>AIOS_TYPE_TRACE</code>	print <code>[rtype] Class#method = tag</code> on every reply ; used by the differential test (<code>scripts/type_runtime_diff.py</code>) that compares the static return type with the type actually returned
<code>AIOS_QUIET</code>	suppress informational messages from the checker and the REPL
<code>AIOS_STRICT_PROTOCOL</code>	warn about the “unfinished” part when the steps of a session cannot be laid out statically. Silent by default (§16)
<code>AIOS_STRICT_RESOURCE</code>	report acquire calls whose name is not a string literal; the order cannot be checked there (§14)
<code>AIOS_SHOW_LEVELS</code>	print the inferred obligation levels and the global resource order
<code>AIOS_LAX_WAIT</code>	downgrade the detection of circular waits from an error to a warning. The default is an error
<code>AIOS_LAX_ACTOR</code>	do not distinguish actor types (unify <code>actor(A)</code> with <code>actor(B)</code>). The default is to distinguish them
<code>AIOS_LAX_ASSIGN</code>	allow assignment to an undeclared name (the old behaviour). The default is an error
<code>AIOS_MODEL_PROVIDER</code>	the model vendor used by <code>ai_call</code> ; the default is <code>gemin</code>

For instance, to work under the discipline of always writing deadlines:

```
AIOS_STRICT_DEADLINE=1 ./src/abclrepl_thread -q -f run.repl
```

41 Common pitfalls

- **now is an expression.** `now x.m();` is a syntax error; always receive it, as in `var r = now x.m() timeout 100 else 0;`.
- **The target of send is a variable**, not a class name.
- **now self.m() cannot be written** (it would be a self-deadlock, so the grammar forbids it). To factor out a computation, move it to another actor.
- **You cannot assign an actor to `var x = 0`; later** (it has been inferred as `int`). Create it as `var x = new Foo();` from the start.
- **String concatenation is ++**; `+` is numeric only.
- **Fields come before methods.**
- **Write a deadline on every wait.** Otherwise you get a warning, and under `AIOS_STRICT_DEADLINE=1` it does not pass at all.
- **Effects are checked once you declare them** (gradual). Declare nothing and nothing is said; declare something and the difference from the body is pointed at.

42 Present limits

- Splitting a wait into **future** + **await** escapes effect propagation (the “known hole” in §11). Fixing it requires carrying effects in the **future** type.
- There is no variable declaration with a type annotation, **var** *x* : *T* = *e*;
- **sender** is **any**, so **send** *sender.m(...)* checks neither the existence of the method nor its types.
- The result of a remote send (**remote(...)**) stays **any**. There is no mechanism for reading the interface description of the peer node.
- Method parameters are monomorphic; polymorphic methods cannot be written.
- There is no implicit conversion between **int** and **float**, and *a + b* with both operands unbound needs an annotation.
- There are no array literals [...]; build arrays with **array_empty** and **array_push**.
- A reply destination (**replyto**) **cannot be kept in a field**, though it can be passed as an argument (§13). A field’s type comes from its initializer, so **var** *w* = 0; is **int** and *w* = **replyto** is a type mismatch.
- The global resource order can only be followed for **acquire** calls whose name is a **string literal** (§14).
- The static session check reaches only as far as **synchronous** callees (§16).
- Termination is not guaranteed. Deadlock freedom means “nothing gets stuck”, not “everything finishes”. **while** 1 > 0 **do** {} does not stop, but it is not a wait.

43 Changes from the previous edition

What changed since the previous edition (2026-07-30).

43.1 Removed from the language

	Content
become	removed from the language . ABCL/1 does not have it; once restricted enough to keep types sound it becomes almost the same as writing a state field and a branch; and not one of 543 real programs used it. Write behaviour replacement with a state field and if

43.2 Added

None of these break existing programs. Every annotation may be omitted, and omitting it means it is inferred.

	Content	Section
Failure type	a wait with no else returns result $\langle\tau\rangle$	§12
First-class reply	replyto / answer / parameter type reply , treated linearly	§13
Resource order	the acquire / release pairs, and the global order read from their nesting	§14
Obligation levels	$\mathcal{O}n$, inferred if omitted; also enforced across node borders	§15
Session types	the type checker reads the run-time protocol declarations	§16
The select discipline	a mandatory deadline, and a check that a sender exists	§17
Mesh deployment	ship the source and type-check it at the destination before instantiating	§18

43.3 Made stricter

	Content
Assignment to undeclared names	now rejected . Previously a new variable was silently created, so a misspelled field name produced no error at all (AIOS_LAX_ASSIGN=1 restores the old behaviour)
Actor types	actor(A) and actor(B) are now distinguished (AIOS_LAX_ACTOR=1 restores the old behaviour)
reply totality	a method waited on by now/await must reply on every path, whatever its return type

43.4 The file extension

The source extension is **.aipl** (**.abcl** has been retired completely). The implementation does not look at the extension, so old files still load, but write new ones as **.aipl**.

43.5 Core and extensions — the language has two layers

There are three implementations, but they are **not the same language**.

Layer	Implemented by	Content
Core	all three	the specification this guide describes
Extensions	the Python implementation only	record types, row polymorphism, refinement types via where , CSP channels, ownership (pub/move), tuples, match , saga , scope { future ... }

This guide specifies the core only. Which layer a program belongs to can be determined mechanically with `tools/classify_corpus.sh` (see `docs/AIPL_LAYERS.md`).

44 Related documents

- `docs/aipl_paper5_ja.pdf` — **the paper (5th edition, latest)**. The current specification, and what was taken from (or left out of) ABCL/1 and Kobayashi’s type systems, with the reasons and the trade-offs.
- `coq/AIPLSoundness2.v` — **the machine-checked proof (2nd version)**, in Coq/Rocq. For the core *including* **await**, type soundness, type safety and deadlock freedom are proved with no axioms. The header also discusses the **largest syntax** for which all three hold at once.

- `coq/AIPLSoundness.v` — the 1st version (deadlock freedom only for the fragment without `await`).
- `docs/AIPLLAYERS.md` — **AIPL has two layers**: the definitions of core (all three implementations) and extensions (Python only), and the classification by `tools/classify_corpus.sh`.
- `docs/aipl_reply_inference_ja.pdf` — return type inference from `reply`, the rationale for annotations, and measurements.
- `docs/aipl_vs_abcl1_ja.pdf` — a comparison with ABCL/1.
- `docs/aipl_abcl1_features_ja.pdf` — the features inherited from ABCL/1.
- `docs/aipl_kobayashi_synthesis_ja.pdf` — a design proposal for taking in the results of Kobayashi's work.
- `docs/ABCLCP_TYPE_SOUNDNESS_REPORT.md` — type soundness (Markdown).
- `docs/AIOS_LANGUAGE_DESIGN.md`, `docs/CAPABILITIES.md` — the design as AIOS and the list of capabilities.
- `docs/samples/reply_inference/README.md` — what the negative samples are for.